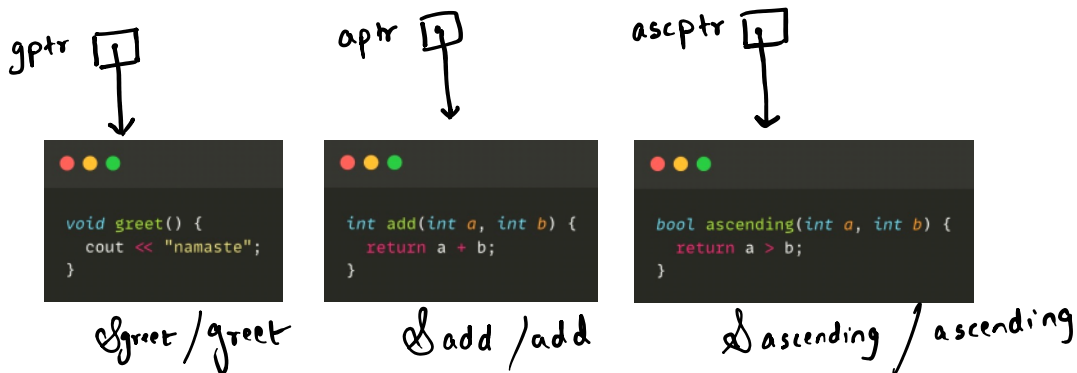
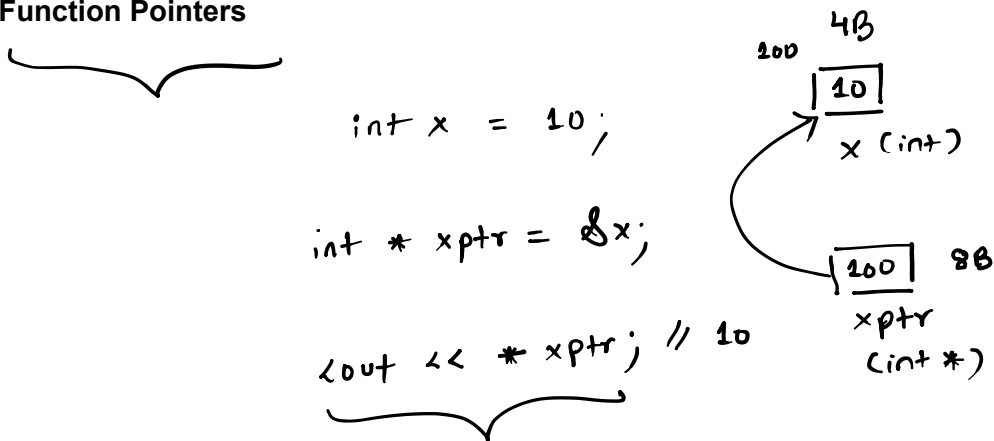


Function Pointers



`void (*gptr)() = &greet;`
`int (*aptr)(int, int) = &add;`
`bool (*ascptr)(int, int) = &ascending;`
`(*gptr)();`
`on`
`greet();`
`(*aptr)(5, 7);`
`or`
`add(5, 7);`
`(*ascptr)(3, 2);`
`or`
`ascending(3, 2);`

Callback Functions

Functions used as arguments to another function are called as **Callback Functions**.

```

int arr[]
void bubbleSort(int* arr, int n) {
    for(int i=1; i<n; i++) {
        for(int j=0; j<n-i; j++) {
            if(arr[j] > arr[j+1]) {
                swap(arr[j], arr[j+1]);
            }
        }
    }
}

```

`int arr[] = { 50, 20, 10, 30, 40 }`
`int n = 5;`
`bubbleSort(arr, n)`
`or`
`&arr[0]`
`10 20 30 40 50`
`arr[j] > arr[j+1]`

```

    }
  }
}

```

arr[0]

10 20 30 40 50
50 40 30 20 10

$arr[j] > arr[j+1]$
 $arr[j] < arr[j+1]$

```

    arr[j] arr[j+1]
bool ascending(int a, int b) {
    return a > b;
}
    arr[j] > arr[j+1]
void bubbleSort(int* arr, int n) {
    for(int i=1; i<n; i++) {
        for(int j=0; j<n-i; j++) {
            if(ascending(arr[j], arr[j+1])) {
                swap(arr[j], arr[j+1]);
            }
        }
    }
}

```

```

    arr[j] arr[j+1]
bool descending(int a, int b) {
    return a < b;
}
    arr[j] < arr[j+1]
void bubbleSort(int* arr, int n) {
    for(int i=1; i<n; i++) {
        for(int j=0; j<n-i; j++) {
            if(descending(arr[j], arr[j+1])) {
                swap(arr[j], arr[j+1]);
            }
        }
    }
}

```

$\text{bubbleSort}(\underline{\text{arr}}, \underline{n}, \&\text{ascending});$
 $\text{bubbleSort}(\text{arr}, n, \&\text{descending});$

```

bool ascending(int a, int b) {
    return a > b;
}

bool descending(int a, int b) {
    return a < b;
}

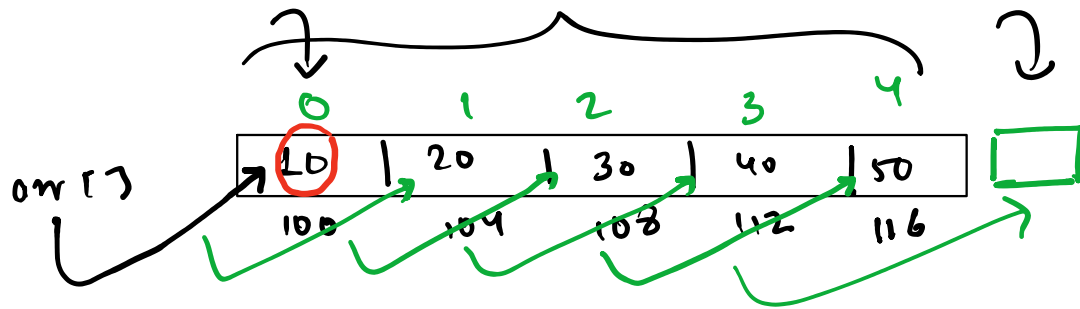
void bubbleSort(int* arr, int n, bool (*f)(int, int)) {
    for(int i=1; i<n; i++) {
        for(int j=0; j<n-i; j++) {
            if(f(arr[j], arr[j+1])) { // (*f)(arr[j], arr[j+1])
                swap(arr[j], arr[j+1]);
            }
        }
    }
}

```

$\text{bubbleSort}(\text{arr}, n, \text{ascending})$
 $\text{bubbleSort}(\text{arr}, n, \text{descending})$

```
int arr[] = { 50, 20, 10, 30, 40 }
```

```
int n = 5; 20B
```



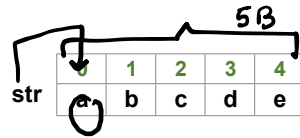
```
sort ( arr , arr+n )
```

[first, last)

```
reverse ( arr, arr+n ) [0, n)
```

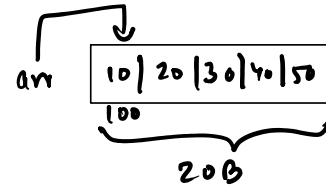
⇒ Introduction to Character Arrays

A character array is a **linear** data structure used to store a **sequence** of characters.



```
int arr[] = {10, 20, 30, 40, 50};
```

```
cout << arr; //100
```



```
for (int i=0; i<n; i++)
    cout << arr[i];
```

```
char str[] = {'a', 'b', 'c', 'd', 'e'};
```

```
cout << str; // abcde garbage
```

```
char str[] = {'a', 'b', 'c', 'd', 'e', '\0'};
```

```
cout << str; // abcde
```

A character array must always be **terminated** with the null character i.e. `'\0'`.

Reading Input into a Character Array

constraints

$n \leq 100$

```
char str[101];
```

```
cin >> str; // '\0' is
            automatically added
```

```
cout << str;
```

based on constraints
and we are choosing
size as 101 to accommodate
the null char when
string len is actually
100

Length of a String

Given a string represented as a character array, design an algorithm to compute its **length** - we define the length of a string as the **number** of characters it contains excluding the null character.

Input : "coding"
Output : 6

"hello"

5

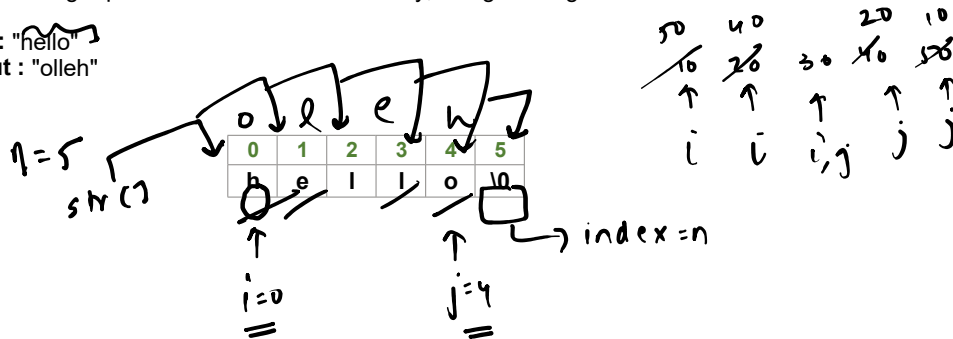
0	1	2	3	4	5	6
c	o	d	i	n	g	\0

cnt = 0
+
+
+
+
+
+ 6

Reverse a String

Given a string represented as a character array, design an algorithm to **reverse** it.

Input : "hello"
Output : "olleh"



→ strrev (—) ← cstring
→ reverse (str, str + n) ← algorithm

Compare Strings

Given a two strings **s1** and **s2** represented as a character arrays, design an algorithm to **compare** them such that output

- 0 if **s1** is equal to **s2**

Given a two strings **s1** and **s2** represented as a character arrays, design an algorithm to **compare** them such that output

- 0 if **s1** is equal to **s2**
- 1 if **s1** is greater than **s2**
- -1 if **s1** is less than **s2**

Handwritten examples of string comparison:

- s1 = "abc", s2 = "abc" → equal / 0**
- s1 = "abcd", s2 = "abcdef" → s2 > s1**
- s1 = "adc", s2 = "abc" → s1 > s2 / 1**
- s1 = "xyz", s2 = "xzy" → s2 > s1 / -1**

[HW] Copy Strings

Given a two strings **s1** and **s2** represented as a character arrays, design an algorithm to **copy** the contents of **s2** into **s1**.

note : assume length of **s2** <= length of **s1**.

Handwritten code snippet:

```
strcpy (s1, s2)
```

[HW] Concatenate Strings

Given a two strings **s1** and **s2** represented as a character arrays, design an algorithm to **concatenate / append** **s2** to **s1**.

note : assume **s1** has enough capacity to accommodate characters of both **s1** & **s2**.

Handwritten code snippet:

```
s1 = "abc";
s2 = "def";
strcat (s1, s2) ; // concatenates s2 to s1
```

cont 21 S1; // abcdef

String Tokenization

Given a string represented as a character array, design an algorithm to **tokenize** it based on a **delimiting** string also represented as a character array.

Tokenization of a string is a process of **splitting** a string into **tokens** (substrings) such that consecutive tokens are separated by one or more delimiting characters.

Example

Input : "abc.def.ghi" and "."

Output : "abc", "def", "ghi"

